The Collections Framework (java.util)

The **java.util package** is an important package in Java that provides many useful classes and interfaces. It supports different functionalities such as:

- Generating pseudorandom numbers
- Managing date and time
- Observing events
- Manipulating sets of bits
- Tokenizing strings
- Handling formatted data

One of the most powerful parts of java.util is the **Collections Framework**.

What is the Collections Framework?

The **Collections Framework** is a sophisticated hierarchy of **interfaces and classes** used for **managing groups of objects**.

It provides advanced technology for storing and manipulating data efficiently. Because of its importance, it deserves special attention from programmers.

Components of java.util

The java.util package contains:

- Many **classes** (such as ArrayList, HashMap, TreeSet, LinkedList, etc.)
- Many **interfaces** (such as Collection, List, Set, Map, Queue, Iterator, etc.)

Collections were **introduced in J2SE 1.2**. Before this, Java used classes like:

- Dictionary
- Vector
- Stack
- Properties

These early classes were useful but:

- They did not follow a common structure
- They were not easily extendable
- They lacked a unified design

The Collections Framework was created to solve these problems.

SBC Karkala

Goals of the Collections Framework

 High Performance
Provides efficient implementations of data structures like dynamic arrays, linked lists, trees, and hash tables.

 Interoperability
Different types of collections work in a similar and consistent manner.

 Easy Extension
Built on standard interfaces, allowing easy extension and custom implementation.

 Integration with Arrays
Allows standard arrays to work within the Collections Framework.

Benefits of Using the Collections Framework

 Provides **high-performance implementations** for dynamic arrays, linked lists, trees, and hash tables.

 Allows different types of collections to work in a **similar manner and with a high degree of interoperability**.

 Makes **extending and/or adapting a collection easy**.

 Provides **standard implementations** such as LinkedList, HashSet, and TreeSet that may be used as-is.

 Provides **algorithms that operate on collections**, defined as static methods within the Collections class.

 Allows the **integration of standard arrays into the Collections Framework**.

 Provides a **better way of doing several things** compared to earlier utility classes.

Important Collection Classes with Purpose

Class	Purpose
ArrayList	Dynamic array used to store ordered elements.
LinkedList	Doubly linked list for efficient insertion and deletion.
HashSet	Stores unique elements using hashing.
TreeSet	Stores sorted unique elements.
HashMap	Stores key/value pairs using hashing.
TreeMap	Stores sorted key/value pairs.
Stack	Implements last-in, first-out (LIFO) structure.
Vector	Dynamic array (legacy class).

SBC Karkala

Recent Changes to Collections

1. Generics Fundamentally Changed the Collections Framework

- The entire Collections Framework was **reengineered for generics**.
- All collections are now **generic**.
- Generics add **type safety**.
- Before generics, collections stored **Object references**, so any type could be stored.
- This could cause **run-time type mismatch errors**.
- With generics, the **type of data can be explicitly stated**.
- Older code must be rewritten to remove **compiler warnings**.

2. Autoboxing Facilitates Primitive Types

- A collection can store only references, not primitive values.
- Earlier, primitives had to be manually boxed into wrapper classes.
- Retrieved values had to be manually unboxed using casting.
- With autoboxing/unboxing, Java automatically performs boxing and unboxing.
- No need to manually perform these operations.

3. The For-Each Style for Loop

- All collection classes now implement the Iterable interface.
- Collections can be cycled through using the for-each loop.
- Earlier, iteration required manually using an iterator.
- In many cases, iterator-based loops can now be replaced by for-each loops.

The Collection Interfaces

The Collections Framework defines several interfaces.
These interfaces form the foundation of all collection classes.
Concrete classes only provide different implementations of these interfaces.

Main Collection Interfaces

Interface	Purpose
Collection	Top of the collections hierarchy. Works with groups of objects.
List	Handles ordered sequences (lists of objects).
Set	Handles sets that contain unique elements.
SortedSet	Handles sorted sets.
NavigableSet	Extends SortedSet for closest-match searches.
Queue	Handles special lists where elements are removed from the head.
Deque	Handles double-ended queues.

SBC Karkala

Other Important Interfaces

- **Comparator → Defines how two objects are compared.**
- **Iterator / ListIterator → Used to access elements one at a time.**
- **RandomAccess → Indicates efficient random access to elements.**

Modifiable and Unmodifiable Collections

- **Some methods are optional.**
- **Collections that allow changes are called modifiable.**
- **Collections that do not allow changes are unmodifiable.**
- **If modification is attempted on an unmodifiable collection → UnsupportedOperationException is thrown.**
- **All built-in collections are modifiable.**

The Collection Interface

- It is the foundation of the Collections Framework.
- It must be implemented by any class that defines a collection.
- It is a generic interface: **interface Collection<E>**
- E specifies the type of objects stored.
- It extends Iterable, so all collections can use the for-each loop.

Important Methods of Collection

Adding Elements

- **add()** → Adds an element.
- **addAll()** → Adds all elements from another collection.

Removing Elements

- **remove()** → Removes one element.
- **removeAll()** → Removes a group of elements.
- **retainAll()** → Keeps only specified elements.
- **clear()** → Removes all elements.

Checking Elements

- **contains()** → Checks if object exists**.**
- **containsAll()** → Checks if collection contains another collection.
- **isEmpty()** → Checks if collection is empty.
- **size()** → Returns number of elements.

Converting to Array

- **toArray()** → Converts collection to array.

Comparing Collections

- **equals()** → Compares two collections.

Iteration

- **iterator()** → Returns an iterator to access elements.

SBC Karkala

The List Interface

- The List interface extends Collection.
- It stores a sequence of elements.
- Elements are accessed using a zero-based index.
- A list can contain duplicate elements.
- It is a generic interface: **interface List<E>**
- E specifies the type of objects stored in the list.

- **Elements can be inserted or accessed by position.**
- **Many methods throw:**
  - UnsupportedOperationException → if list cannot be modified
  - ClassCastException → if incompatible object is added
  - IndexOutOfBoundsException → if invalid index is used
  - NullPointerException → if null is not allowed
  - IllegalArgumentException → if invalid argument is used

Methods in List

- **void add(int index, E obj)**
  Inserts obj at the given index in the list.
- **E get(int index)**
  Returns the object at the specified index.
- **E set(int index, E obj)**
  Assigns obj to the location specified by index within the invoking list
- **int indexOf(Object obj)**
  Returns the index of the first occurrence of the specified obj. If obj is not an element of the list, –1 is returned.
- **List<E> subList(int start, int end)**
  Returns a portion of the list between the specified beginning and ending indexes.
- **E remove(int index)**
  Removes the element at the specified index from the list and returns the deleted element.
- **int lastIndexOf(Object obj)**
  Returns the index of the last instance of obj in the invoking list. If obj is not an element of the list, –1 is returned.

The Set Interface

- Set extends Collection.
- It represents a collection that does not allow duplicate elements.
- If a duplicate element is added, add() returns false.
- It does not define any new methods of its own.
- It is a generic interface: **interface Set<E>**
- E specifies the type of objects stored in the set.

SBC Karkala

The SortedSet Interface

- SortedSet extends Set.
- It represents a set that is sorted in ascending order.
- It is also a generic interface: **interface SortedSet<E>**

Important Methods of SortedSet

1. **Comparator<? super E> comparator()**
   Returns the comparator used for sorting.
   Returns null if natural ordering is used.
2. **E first()**
   Returns the first (smallest) element.
3. **E last()**
   Returns the last (largest) element.
4. **SortedSet<E> subSet(E start, E end)**
   Returns elements between start and end–1.
5. **SortedSet<E> headSet(E end)**
   Returns elements less than end.
6. **SortedSet<E> tailSet(E start)**
   Returns elements greater than or equal to start**.**

Possible Exceptions

- NoSuchElementException → If set is empty.
- ClassCastException → If object is incompatible.
- NullPointerException → If null not allowed.
- IllegalArgumentException → If invalid argument is used.

NavigableSet

- **NavigableSet** was added in **Java SE 6**.
- It **extends SortedSet**.
- It supports retrieval of elements based on the **closest match** to a given value.
- It is a generic interface:

interface NavigableSet<E>

- E specifies the type of objects stored.

NavigableSet provides advanced searching and navigation methods such as **ceiling(), floor(), higher(), lower(), pollFirst(), and pollLast()**, which help retrieve elements based on closest match operations.

Methods of NavigableSet
E ceiling(E obj)
Syntax: E ceiling(E obj)
Purpose:
Returns the smallest element **greater than or equal to** obj.
Returns null if no such element exists.

SBC Karkala

E floor(E obj)
Syntax: E floor(E obj)
Purpose:
Returns the largest element **less than or equal to** obj.
Returns null if no such element exists.

E higher(E obj)
Syntax: E higher(E obj)
Purpose:
Returns the smallest element **strictly greater than** obj.
Returns null if no such element exists.

E lower(E obj)
Syntax: E lower(E obj)
Purpose:
Returns the largest element **strictly less than** obj.
Returns null if no such element exists.

E pollFirst()
Syntax: E pollFirst()
Purpose:
Returns and removes the **first (smallest)** element.
Returns null if the set is empty.

E pollLast()
Syntax: E pollLast()
Purpose:
Returns and removes the **last (largest)** element.
Returns null if the set is empty.

The Queue Interface

- **Queue** extends **Collection**.
- It represents a **queue data structure**.
- It is generally **First-In, First-Out (FIFO)**.
- It is a generic interface: **interface Queue<E>**
- E specifies the type of objects stored in the queue.
- Elements are **removed only from the head**.
- Some queues may have different ordering criteria.
- Some queues may have **fixed length**.

Methods of Queue
E element()
Purpose:
Returns the head element.
Does **not remove** it.
Throws **NoSuchElementException** if empty.

SBC Karkala

boolean offer(E obj)
Purpose:
Attempts to add an element to the queue.
Returns **true** if successful, otherwise **false**.

E peek()
Purpose:
Returns the head element.
Does **not remove** it.
Returns **null** if queue is empty.

E poll()
Purpose:
Returns and **removes** the head element.
Returns **null** if queue is empty.

E remove()
Purpose:
Removes and returns the head element.
Throws **NoSuchElementException** if empty.

The Deque Interface

- **Deque** stands for **Double-Ended Queue**.
- It was added in **Java SE 6**.
- It extends **Queue**.
- It allows insertion and removal of elements from **both ends**.
- It can work as:
  - **FIFO queue**
  - **LIFO stack**
- It is a generic interface: **interface Deque<E>**
- E specifies the type of elements stored.

Methods of Deque
void addFirst(E obj)
Purpose:
Adds an element to the **head (front)** of the deque.
Throws **IllegalStateException** if full.

void addLast(E obj)
Purpose:
Adds an element to the **tail (end)** of the deque.
Throws **IllegalStateException** if full.

E removeFirst()
Purpose:
Removes and returns the **first element**.
Throws **NoSuchElementException** if empty.

SBC Karkala

E removeLast()
Purpose:
Removes and returns the **last element**.
Throws **NoSuchElementException** if empty.

void push(E obj)
Purpose:
Adds an element to the **head** of the deque.
Used to implement **stack behavior**.

E pop()
Purpose:
Removes and returns the element at the **head**.
Used to implement **stack behavior**.
Throws **NoSuchElementException** if empty.

The Collection Classes
After understanding the collection interfaces, the next step is to study the standard classes that implement them.

- Some classes provide complete implementations and can be used directly.
- Some are abstract classes that provide skeletal implementations.
- None of the collection classes are synchronized.
- However, synchronized versions can be obtained.

Class	Description
AbstractCollection	Implements most of the **Collection** interface.
AbstractList	Extends **AbstractCollection** and implements most of the **List** interface.
AbstractQueue	Extends **AbstractCollection** and implements parts of the **Queue** interface.
AbstractSequentialList	Extends **AbstractList** for use by a collection that uses sequential rather than random access of its elements.
LinkedList	Implements a linked list by extending **AbstractSequentialList**.
ArrayList	Implements a dynamic array by extending **AbstractList**.
ArrayDeque	Implements a dynamic double-ended queue by extending **AbstractCollection** and implementing the **Deque** interface. (Added by Java SE 6.)
AbstractSet	Extends **AbstractCollection** and implements most of the **Set** interface.
EnumSet	Extends **AbstractSet** for use with **enum** elements.
HashSet	Extends **AbstractSet** for use with a hash table.
LinkedHashSet	Extends **HashSet** to allow insertion-order iterations.
PriorityQueue	Extends **AbstractQueue** to support a priority-based queue.
TreeSet	Implements a set stored in a tree. Extends **AbstractSet**.

karkala

The ArrayList Class

The **ArrayList** class extends **AbstractList** and implements the **List** interface.
It is a **generic class**, declared as:
class ArrayList<E>
Here, **E** specifies the type of objects that the list will hold.

ArrayList supports **dynamic arrays that can grow as needed**.
In Java:

- Standard arrays are of **fixed length**.
- Once created, arrays **cannot grow or shrink**.
- The size must be known in advance.

However, in many situations, the exact size needed is **not known at compile time**.
To solve this problem, the Collections Framework provides **ArrayList**.
An **ArrayList is essentially a variable-length array of object references**.
It can dynamically increase or decrease in size.

Key Features of ArrayList

1. Supports **dynamic resizing**.
2. Stores **object references only**.
3. Works with primitive types using **autoboxing**.
4. Maintains **insertion order**.
5. Allows **duplicate elements**.
6. Provides **index-based access** (zero-based index).

Constructors of ArrayList

ArrayList defines the following constructors:

1. Default Constructor

ArrayList()
Creates an empty ArrayList.

2. Constructor with Collection

ArrayList(Collection<? extends E> c)
Creates an ArrayList that contains the elements of another collection.

3. Constructor with Initial Capacity

ArrayList(int capacity)
Creates an ArrayList with the specified initial capacity.

Capacity Handling in ArrayList

1. ensureCapacity()

void ensureCapacity(int cap)
Purpose:

- Increases the capacity of the ArrayList to at least cap.
- Used to manually expand internal storage.

SBC Karkala

2. trimToSize()
void trimToSize()
Purpose:

- Reduces the capacity to match the current number of elements.
- Removes unused internal space.

Important Methods Used in ArrayList
add()
Adds an element to the list.
boolean add(E obj)

add(int index, E obj)
Inserts an element at the specified position.

remove(Object obj)
Removes the specified object.

remove(int index)
Removes the element at the specified index.

size()
Returns the number of elements in the list.

toArray()
Converts the ArrayList into an array.
Two forms:
Object[] toArray()
<T> T[] toArray(T array[])

Autoboxing in ArrayList
ArrayList stores only object references.
Primitive values are automatically converted into wrapper objects.
Example:

```
ArrayList<Integer> al = new ArrayList<>();
al.add(10);   // int automatically boxed to Integer
```

Example Program

```
import java.util.*;
class ArrayListDemo {
   public static void main(String args[]) {
      ArrayList<String> al = new ArrayList<String>();
      System.out.println("Initial size of al: " + al.size());
      al.add("C");
      al.add("A");
      al.add("E");
      al.add("B");
```

SBC Karkala

```
        al.add("D");
        al.add("F");
        al.add(1, "A2");

        System.out.println("Size of al after additions: " + al.size());
        System.out.println("Contents of al: " + al);

        al.remove("F");
        al.remove(2);

        System.out.println("Size of al after deletions: " + al.size());
        System.out.println("Contents of al: " + al);
    }
}
```

Output

```
Initial size of al: 0
Size of al after additions: 7
Contents of al: [C, A2, A, E, B, D, F]
Size of al after deletions: 5
Contents of al: [C, A2, E, B, D]
```

The LinkedList Class

The **LinkedList** class:

- Extends **AbstractSequentialList**
- Implements **List**, **Deque**, and **Queue** interfaces
- Provides a **linked-list data structure**

It is a **generic class**, declared as:

```
class LinkedList<E>
```

Here, **E** specifies the type of objects that the list will hold.

Constructors of LinkedList

LinkedList has two constructors:

1. Default Constructor

```
LinkedList()
```

Creates an empty linked list.

2. Constructor with Collection

```
LinkedList(Collection<? extends E> c)
```

Creates a linked list initialized with the elements of collection c.

Important Features

Because LinkedList implements: **List, Deque, Queue**

You can use all methods defined by these interfaces.

SBC Karkala

Adding Elements

Add to end:

add(E obj)

addLast(E obj)

offerLast(E obj)

Add to beginning:

addFirst(E obj)

offerFirst(E obj)

Add at specific position:

add(int index, E obj)

Getting Elements

Get first element:

getFirst()

peekFirst()

Get last element:

getLast()

peekLast()

Get element by index:

get(int index)

Removing Elements

Remove first element:

removeFirst()

pollFirst()

Remove last element:

removeLast()

pollLast()

Remove specific element:

remove(Object obj)

Remove by index:

remove(int index)

Modifying Elements

Change value at index:

set(int index, E obj)

To update a value:

1. Use get(index) to retrieve it.
2. Use set(index, newValue) to replace it.

Example Program

```
import java.util.*;
class LinkedListDemo {
   public static void main(String args[]) {
      // Create a linked list
      LinkedList<String> ll = new LinkedList<String>();
```

SBC Karkala

```
        // Add elements
        ll.add("F");
        ll.add("B");
        ll.add("D");
        ll.add("E");
        ll.add("C");
        ll.addLast("Z");
        ll.addFirst("A");
        ll.add(1, "A2");

        System.out.println("Original contents of ll: " + ll);

        // Remove elements
        ll.remove("F");
        ll.remove(2);

        System.out.println("Contents of ll after deletion: " + ll);

        // Remove first and last elements
        ll.removeFirst();
        ll.removeLast();

        System.out.println("ll after deleting first and last: " + ll);

        // Get and set value
        String val = ll.get(2);
        ll.set(2, val + " Changed");

        System.out.println("ll after change: " + ll);
    }
}
```

Output

Original contents of ll: [A, A2, F, B, D, E, C, Z]
Contents of ll after deletion: [A, A2, D, E, C, Z]
ll after deleting first and last: [A2, D, E, C]
ll after change: [A2, D, E Changed, C]

HashSet Class

- HashSet extends AbstractSet
- Implements the Set interface
- Uses a **hash table** for storage
- It is a **generic class**

Declaration: **class HashSet<E>**

- E → Type of elements stored in the set

SBC Karkala

Hashing Concept

- A hash table stores data using a method called **hashing**
- The key is converted into a **hash code**
- The hash code determines the index where data is stored
- This transformation happens automatically
- We cannot directly see or access the hash code

Advantage of Hashing

The execution time of the following methods remains **constant**, even for large sets:

- add()
- contains()
- remove()
- size()

Constructors of HashSet

1. HashSet()
   → Creates default hash set
2. HashSet(Collection<? extends E> c)
   → Creates hash set using elements of collection c
3. HashSet(int capacity)
   → Sets initial capacity
   → Default capacity = **16**
4. HashSet(int capacity, float fillRatio)
   → Sets capacity and fill ratio (load factor)
   → Fill ratio must be between **0.0 and 1.0**
   → Default fill ratio = **0.75**

Fill Ratio (Load Factor)

- Determines how full the set can be before resizing
- When:

number of elements > capacity × fillRatio

The hash set expands

Additional Methods

- HashSet does not define any new methods
- It uses methods from its superclasses and interfaces

Important Note About Ordering

- HashSet does **not guarantee order**
- Elements are not stored in sorted order
- Output may vary
- If sorted storage is required → use TreeSet

SBC Karkala

Example Program

```
import java.util.*;
class HashSetDemo {
   public static void main(String args[]) {
      HashSet<String> hs = new HashSet<String>();
      hs.add("B");
      hs.add("A");
      hs.add("D");
      hs.add("E");
      hs.add("C");
      hs.add("F");
      System.out.println(hs);
   }
}
```

Output (may vary)

[D, A, F, C, B, E]

- Elements are not in sorted order
- Output order may change each time

LinkedHashSet Class

Definition

- LinkedHashSet extends HashSet
- Does not add any new methods or members
- It is a **generic class**

Declaration: **class LinkedHashSet<E>**

- E → Type of elements stored

Constructors

- Constructors are same as HashSet
- Parallel to HashSet constructors

Important Feature

LinkedHashSet maintains a **linked list** of elements

- Elements are stored in the **order in which they were inserted**
- This is called **insertion-order iteration**

Iteration Behavior

When iterating using:

- Iterator
- for-each loop
- toString()

Elements are returned in the **same order as insertion**

SBC Karkala

Difference from HashSet

HashSet	LinkedHashSet
No guaranteed order	Maintains insertion order
Output order may vary	Output order same as insertion

Example (Replace HashSet with LinkedHashSet)

```
import java.util.*;
class LinkedHashSetDemo {
   public static void main(String args[]) {
      LinkedHashSet<String> hs = new LinkedHashSet<String>();
      hs.add("B");
      hs.add("A");
      hs.add("D");
      hs.add("E");
      hs.add("C");
      hs.add("F");
      System.out.println(hs);
   }
}
```

Output

[B, A, D, E, C, F]

- Elements appear in **insertion order**
- Order is preserved

TreeSet Class

Definition

- TreeSet extends AbstractSet
- Implements the NavigableSet interface
- Uses a **tree** for storage
- Stores elements in **sorted, ascending order**
- Fast access and retrieval
- Suitable for storing large amounts of sorted data

Generic Declaration : **class TreeSet<E>**

- E → Type of objects stored in the set

Constructors

1. TreeSet()
   → Creates empty TreeSet
   → Sorted in ascending order using natural ordering
2. TreeSet(Collection<? extends E> c)
   → Creates TreeSet containing elements of collection c
3. TreeSet(Comparator<? super E> comp)
   → Creates empty TreeSet sorted using given comparator
4. TreeSet(SortedSet<E> ss)
   → Creates TreeSet containing elements of another SortedSet

SBC Karkala

Example Program

```
import java.util.*;
class TreeSetDemo {
   public static void main(String args[]) {
      TreeSet<String> ts = new TreeSet<String>();
      ts.add("C");
      ts.add("A");
      ts.add("B");
      ts.add("E");
      ts.add("F");
      ts.add("D");
      System.out.println(ts);
   }
}
```

Output

[A, B, C, D, E, F]

- Elements are automatically arranged in **sorted order**

NavigableSet Features

Since TreeSet implements NavigableSet, its methods can be used.

Example:

```
System.out.println(ts.subSet("C", "F"));
```

Output

[C, D, E]

- Returns elements from **C (inclusive)** to **F (exclusive)**

PriorityQueue Class

Definition

- PriorityQueue extends AbstractQueue
- Implements the Queue interface
- Elements are stored based on **priority**
- Priority is determined by a **Comparator**
- It is a **generic class**

Declaration:

```
class PriorityQueue<E>
```

- E → Type of elements stored

Important Features

- It is **dynamic** (grows automatically)
- Elements are ordered based on priority
- By default, elements are arranged in **ascending order**
- The **smallest element is at the head**

SBC Karkala

Constructors

1. PriorityQueue()
   → Creates empty queue
   → Default capacity = **11**
2. PriorityQueue(int capacity)
   → Creates queue with specified initial capacity
3. PriorityQueue(int capacity, Comparator<? super E> comp)
   → Creates queue with specified capacity and comparator
4. PriorityQueue(Collection<? extends E> c)
5. PriorityQueue(PriorityQueue<? extends E> c)
6. PriorityQueue(SortedSet<? extends E> c)
   → These constructors initialize the queue with elements from collection c

In all cases, capacity increases automatically when needed.

Ordering Rule

- If **no comparator** is given → natural ordering is used
- Natural ordering → ascending order
- Smallest element gets highest priority
- If a **custom comparator** is given → ordering changes

Example use case:

- Time stamp based priority
- Oldest item first

---

comparator() Method

Comparator<? super E> comparator()

- Returns the comparator used
- Returns **null** if natural ordering is used

Note About Iteration

- You can iterate using iterator
- But iteration order is **undefined**
- Correct usage is through queue methods:
  - offer() → insert element
  - poll() → remove head element

ArrayDeque Class

Definition

- Added in **Java SE 6**
- ArrayDeque extends AbstractCollection
- Implements the Deque interface
- Does not define any new methods
- Uses a **dynamic array**
- No capacity restrictions
- Capacity increases automatically

Generic Declaration **class ArrayDeque<E>**

- E → Type of objects stored

SBC Karkala

Constructors

1. ArrayDeque()
   → Creates empty deque
   → Default capacity = **16**
2. ArrayDeque(int size)
   → Creates deque with specified initial capacity
3. ArrayDeque(Collection<? extends E> c)
   → Creates deque initialized with elements of collection c

In all cases, capacity grows automatically when needed.

---

4 Important Points

- Implements Deque (Double Ended Queue)
- Can be used as:
  - Stack
  - Queue
- Uses dynamic array internally

Example: Using ArrayDeque as Stack

```
import java.util.*;
class ArrayDequeDemo {
   public static void main(String args[]) {
      ArrayDeque<String> adq = new ArrayDeque<String>();
      adq.push("A");
      adq.push("B");
      adq.push("D");
      adq.push("E");
      adq.push("F");
      System.out.print("Popping the stack: ");
      while(adq.peek() != null)
         System.out.print(adq.pop() + " ");
      System.out.println();
   }
}
```

Output

Popping the stack: F E D B A

Explanation of Output

- push() → adds elements to top
- pop() → removes elements from top
- Stack follows **LIFO (Last In First Out)**
- Last inserted element (F) is removed first

SBC Karkala

EnumSet Class

Definition

- EnumSet extends AbstractSet
- Implements Set
- Used specifically for **enum type elements**
- It is a **generic class**

Declaration:

class EnumSet<E extends Enum<E>>

- E must extend Enum<E>
- Ensures elements belong to a specified enum type

- Only **enum values** can be stored
- Other object types are not allowed

---

3 Constructors

- EnumSet does **not** define constructors
- Objects are created using **factory methods**

---

4 Factory Methods

- Used to create EnumSet objects
- of() method is overloaded multiple times
- Overloading improves efficiency
- Passing fixed number of arguments can be faster than varargs

---

5 Exceptions

All methods may throw:

- NullPointerException

copyOf() and range() may also throw:

- IllegalArgumentException

Accessing a Collection via Iterator

- To cycle through elements in a collection, an **iterator** is used.
- It allows access to elements one at a time.
- An iterator object implements either:
  - Iterator<E>
  - ListIterator<E>

◆ Iterator

An **Iterator** is an object used to traverse (access one by one) the elements of a collection.

- Enables cycling through a collection.
- Can obtain elements.
- Can remove elements.

Declaration: **interface Iterator<E>**

- E specifies the type of objects being iterated.

SBC Karkala

Commonly Used Methods Defined by Iterator

boolean hasNext()

- Checks if more elements are available.
- Returns true if next element exists.

E next()

- Returns the next element.
- Moves the cursor forward.
- Throws NoSuchElementException if no element.

---

3 **void remove()**

- Removes the current element.
- Must be called after next().
- Otherwise throws IllegalStateException.

Example

```
import java.util.*;
class IteratorExample {
   public static void main(String[] args) {
      ArrayList<String> list = new ArrayList<>();
      list.add("A");
      list.add("B");
      list.add("C");
      Iterator<String> itr = list.iterator()
      while (itr.hasNext()) {
         String element = itr.next();
         System.out.print(element + " ");
      }
   }
}
```

Output : A B C

🔹 **ListIterator**

- Extends Iterator.
- Allows bidirectional traversal (forward and backward).
- Allows modification of elements.

Declaration: **interface ListIterator<E>**

- E specifies the type of objects being iterated.

Methods Defined by ListIterator<E>

void add(E obj)

- Inserts obj into the list in front of the element that will be returned by the next call to next().

boolean hasNext()

- Returns true if there is a next element.
- Otherwise, returns false.

SBC Karkala

boolean hasPrevious()

- Returns true if there is a previous element.
- Otherwise, returns false.

E next()

- Returns the next element.
- Throws NoSuchElementException if there is not a next element.

int nextIndex()

- Returns the index of the next element.
- If there is not a next element, returns the size of the list.

E previous()

- Returns the previous element.
- Throws NoSuchElementException if there is not a previous element.

int previousIndex()

- Returns the index of the previous element.
- If there is not a previous element, returns -1.

void remove()

- Removes the current element from the list.
- Throws IllegalStateException if remove() is called before next() or previous().

void set(E obj)

- Assigns obj to the current element.
- This is the element last returned by a call to either next() or previous().

To use an iterator to cycle through the contents of a collection, follow these steps:

1. Obtain an iterator to the start of the collection by calling the collection's iterator( ) method.
2. Set up a loop that makes a call to hasNext( ). Have the loop iterate as long as hasNext( ) returns true.
3. Within the loop, obtain each element by calling next( ).

The For-Each Alternative to Iterators

Usage of For-Each Loop with Collections

The **for-each loop** (enhanced for loop) is a simpler alternative to using an iterator when:

- You do **not** need to modify the collection.
- You do **not** need reverse traversal.
- You only need forward traversal.

◆ **Key Points**

- It works on any collection that implements the **Iterable** interface.
- All collection classes implement Iterable.
- It is shorter and simpler than the iterator-based approach.
- It can only traverse in the **forward direction**.
- It **cannot modify** the contents of the collection.

SBC Karkala

🔹 **Syntax**

```
for (Type variable : collection) {
   // statements
}
```

🔹 **Example Program**

```
import java.util.*;
class ForEachDemo {
   public static void main(String args[]) {
      ArrayList<Integer> vals = new ArrayList<Integer>();
      vals.add(1);
      vals.add(2);
      vals.add(3);
      vals.add(4);
      vals.add(5);
      System.out.print("Original contents of vals: ");
      for(int v : vals)
         System.out.print(v + " ");
      System.out.println();

      int sum = 0;
      for(int v : vals)
         sum += v;
      System.out.println("Sum of values: " + sum);
   }
}
```

🔹 **Output**

```
Original contents of vals: 1 2 3 4 5
Sum of values: 15
```

Storing User-Defined Classes in Collections

🔹 **Key Concept**

- Collections are not limited to built-in classes like String or Integer.
- They can store **any type of object**, including user-defined classes.
- This is one of the main strengths of the Collections Framework.

🔹 **Example: Storing a User-Defined Class in a LinkedList**

A LinkedList is used to store objects of a custom class named Address.

Step 1: User-Defined Class "Address"

- Stores mailing address details.
- toString() is overridden to display object information properly.

Step 2: Storing Objects in LinkedList

SBC Karkala

```
class Address {
   private String name;
   private String street;
   private String city;
   private String state;
   private String code;
   Address(String n, String s, String c, String st, String cd) {
      name = n;
      street = s;
      city = c;
      state = st;
      code = cd;
   }
   public String toString() {
      return name + "\n" + street + "\n" +
            city + " " + state + " " + code;
   }
}

import java.util.*;
class MailList {
   public static void main(String args[]) {
      LinkedList<Address> ml = new LinkedList<Address>();
      ml.add(new Address("J.W. West", "11 Oak Ave",
                  "Urbana", "IL", "61801"));
      ml.add(new Address("Ralph Baker", "1142 Maple Lane",
                  "Mahomet", "IL", "61853"));
      ml.add(new Address("Tom Carlton", "867 Elm St",
                  "Champaign", "IL", "61820"));
      for(Address element : ml)
        System.out.println(element + "\n");
   }
}
```

Output

```
J.W. West
11 Oak Ave
Urbana IL 61801

Ralph Baker
1142 Maple Lane
Mahomet IL 61853

Tom Carlton
867 Elm St
Champaign IL 61820
```

SBC Karkala

The RandomAccess Interface

- RandomAccess is a marker interface (it has no methods).
- It indicates that a collection supports fast index-based access.
- It is used to identify efficient random access at runtime.
- It helps in choosing appropriate looping methods (index-based vs iterator).
- It is implemented by ArrayList.
- It is implemented by Vector.
- It is not implemented by LinkedList.

Working with Maps

A **Map** is an object that stores associations between **keys and values (key–value pairs)**. Given a key, its corresponding value can be retrieved. Both keys and values are objects.

- **Keys must be unique**, but **values can be duplicated**.
- Some maps allow **null keys and null values**, while others do not.

Maps **do not implement the Iterable interface**. Therefore, a map **cannot be traversed using a for-each loop directly**, and an **iterator cannot be obtained directly**. However, a **collection view of the map** can be obtained, which allows traversal using a loop or iterator.

Map Interfaces

The following interfaces support maps:

Interface	Description
Map	Maps unique keys to values.
Map.Entry	Describes an element (key/value pair) in a map. It is an inner interface of Map.
SortedMap	Extends Map and maintains keys in **ascending order**.
NavigableMap	Extends SortedMap and supports retrieval based on **closest-match searches**.

The Map Interface

The **Map interface** maps **unique keys to values**. A key is used to retrieve a stored value.

Map is **generic** and declared as: **interface Map<K, V>**

- **K** – type of keys, **V** – type of values

Maps mainly use two operations:

- **put()** – stores a key and value in the map.
- **get()** – retrieves the value associated with a given key.

The Methods Defined by Map

Method	Description
V put(K k, V v)	Inserts a key–value pair into the map. If the key already exists, the old value is replaced.
V get(Object k)	Returns the value associated with the specified key k. Returns null if the key is not found.
V remove(Object k)	Removes the entry whose key equals k from the map.
boolean containsKey(Object k)	Returns true if the map contains the specified key k; otherwise returns false.
int size()	Returns the number of key–value pairs present in the map.
void clear()	Removes all key–value pairs from the map, making it empty.

Although maps are part of the **Collections Framework**, they are **not collections** because they do not implement the **Collection interface**. However, a **collection-view** of a map can be obtained.
The **entrySet()** method returns a **Set containing all key–value pairs** in the map.
The **keySet()** method returns a **Set containing all the keys** in the map.
The **values()** method returns a **Collection containing all the values** in the map.
These **collection-views allow maps to be integrated with the Collections Framework and enable traversal of map elements**.

The SortedMap Interface
The **SortedMap interface** extends the **Map interface**. It stores the entries in **ascending order based on the keys**.
SortedMap is **generic** and declared as:
interface SortedMap<K, V>

- **K** – type of keys
- **V** – type of values

Some methods may throw exceptions such as:

- **NoSuchElementException** – when the map contains no elements.
- **ClassCastException** – when an incompatible object is used.
- **NullPointerException** – when a null object is used and null values are not allowed.
- **IllegalArgumentException** – when an invalid argument is passed.

SortedMap allows efficient manipulation of **submaps (subsets of a map)**.

Method	Description
Comparator<? super K> comparator( )	Returns the invoking sorted map's comparator. If natural ordering is used for the invoking map, **null** is returned.
K firstKey( )	Returns the first key in the invoking map.
SortedMap<K, V> headMap(K *end*)	Returns a sorted map for those map entries with keys that are less than *end*.
K lastKey( )	Returns the last key in the invoking map.
SortedMap<K, V> subMap(K *start*, K *end*)	Returns a map containing those entries with keys that are greater than or equal to *start* and less than *end*.
SortedMap<K, V> tailMap(K *start*)	Returns a map containing those entries with keys that are greater than or equal to *start*.

TABLE 17-11 The Methods Defined by **SortedMap**

The NavigableMap Interface
The **NavigableMap interface** was introduced in **Java SE 6**. It extends the **SortedMap interface** and supports retrieval of entries based on the **closest match to a given key**.
NavigableMap is **generic** and declared as:
interface NavigableMap<K,V>

- **K** – type of keys
- **V** – type of values associated with the keys

In addition to the methods inherited from **SortedMap**, NavigableMap provides methods that help find entries **closest to a specified key**.

Some methods may throw exceptions such as:

- **ClassCastException** – when an object is incompatible with the keys in the map.
- **NullPointerException** – when a null object is used and null keys are not allowed.
- **IllegalArgumentException** – when an invalid argument is used.

Methods of NavigableMap

Method	Description
Map.Entry<K,V> firstEntry()	Returns the first entry in the map. This is the entry with the **smallest key**.
Map.Entry<K,V> lastEntry()	Returns the last entry in the map. This is the entry with the **largest key**.
K ceilingKey(K obj)	Returns the **smallest key greater than or equal to** the specified key obj.
K floorKey(K obj)	Returns the **largest key less than or equal to** the specified key obj.
K higherKey(K obj)	Returns the **smallest key greater than** the specified key obj.
K lowerKey(K obj)	Returns the **largest key less than** the specified key obj.

The Map.Entry Interface

The Map.Entry interface enables you to work with a map entry. Recall that the entrySet( ) method declared by the Map interface returns a Set containing the map entries. Each of these set elements is a Map.Entry object. Map.Entry is generic and is declared like this:

interface Map.Entry<K, V>

Here, K specifies the type of keys, and V specifies the type of values.

Method	Description
boolean equals(Object *obj*)	Returns **true** if *obj* is a **Map.Entry** whose key and value are equal to that of the invoking object.
K getKey( )	Returns the key for this map entry.
V getValue( )	Returns the value for this map entry.
int hashCode( )	Returns the hash code for this map entry.
V setValue(V *v*)	Sets the value for this map entry to *v*. A **ClassCastException** is thrown if *v* is not the correct type for the map. An **IllegalArgumentException** is thrown if there is a problem with *v*. A **NullPointerException** is thrown if *v* is **null** and the map does not permit **null** keys. An **UnsupportedOperationException** is thrown if the map cannot be changed.

TABLE 17-13 The Methods Defined by **Map.Entry**

The Map Classes

Several classes provide implementations of the map interfaces. The classes that can be used for maps are summarized here:

Class	Description
AbstractMap	Implements most of the **Map** interface.
EnumMap	Extends **AbstractMap** for use with **enum** keys.
HashMap	Extends **AbstractMap** to use a hash table.
TreeMap	Extends **AbstractMap** to use a tree.
WeakHashMap	Extends **AbstractMap** to use a hash table with weak keys.
LinkedHashMap	Extends **HashMap** to allow insertion-order iterations.
IdentityHashMap	Extends **AbstractMap** and uses reference equality when comparing documents.

AbstractMap is a superclass for all concrete map implementations.

WeakHashMap implements a map that uses "weak keys," which allows an element in a map to be garbage-collected when its key is otherwise unused.

HashMap Class

The **HashMap class** extends **AbstractMap** and implements the **Map interface**. It uses a **hash table** to store the map. This allows the execution time of **get()** and **put()** to remain constant even for large sets.

HashMap is a **generic class** and is declared as:

class HashMap<K, V>

Here, **K** specifies the type of keys, and **V** specifies the type of values.

Constructors of HashMap

The following constructors are defined:

HashMap()

HashMap(Map<? extends K, ? extends V> m)

HashMap(int capacity)

HashMap(int capacity, float fillRatio)

- The first form constructs a **default hash map**.
- The second form initializes the hash map using the **elements of m**.
- The third form initializes the **capacity of the hash map** to *capacity*.
- The fourth form initializes both the **capacity and fill ratio** of the hash map using its arguments.

The **default capacity is 16** and the **default fill ratio is 0.75**.

Important Points

- HashMap **implements Map** and **extends AbstractMap**.
- It **does not add any methods of its own**.
- A hash map **does not guarantee the order of its elements**.
- Therefore, the order in which elements are added to a hash map is **not necessarily the order in which they are read by an iterator**.

SBC Karkala

Working of HashMap

A HashMap stores **key–value pairs**.

A **set-view** of the map can be obtained using **entrySet()**.

- **entrySet()** returns a set of elements in the map.
- The keys and values can be accessed using **getKey()** and **getValue()** methods of **Map.Entry**.
- The **put()** method replaces the existing value if the specified key already exists in the map.

Java Program Demonstrating HashMap Usage

```
import java.util.*;
class HashMapExample {
   public static void main(String args[])     {
      // Create a HashMap
      HashMap<String, Integer> hm = new HashMap<String, Integer>();

      // Adding elements to the map
      hm.put("Apple", 10);
      hm.put("Mango", 20);
      hm.put("Orange", 15);
      hm.put("Banana", 25);

      // Display the HashMap
      System.out.println("Elements in HashMap:");
      Set<Map.Entry<String, Integer>> set = hm.entrySet();
      for(Map.Entry<String, Integer> me : set)
      {
         System.out.println(me.getKey() + " : " + me.getValue());
      }
      // Accessing a value using key
      System.out.println("\nValue of Mango: " + hm.get("Mango"));
      // Updating a value
      hm.put("Mango", 30);
      System.out.println("Updated value of Mango: " + hm.get("Mango"));
   }
}
```

Sample Output

```
Elements in HashMap:
Apple : 10
Mango : 20
Orange : 15
Banana : 25

Value of Mango: 20
Updated value of Mango: 30
```

SBC Karkala

Explanation

- A **HashMap hm** is created.
- **put()** inserts key–value pairs.
- **entrySet()** returns the set of map entries.
- **getKey() and getValue()** access keys and values.
- **get()** retrieves the value of a given key.
- **put()** updates the value if the key already exists.

TreeMap Class

The **TreeMap class** extends **AbstractMap** and implements the **NavigableMap interface**. It stores map entries in a **tree structure**. TreeMap stores key–value pairs in **sorted order** and allows **fast retrieval**.

Unlike **HashMap**, **TreeMap guarantees that elements are sorted in ascending order of keys**.

TreeMap is a **generic class** and declared as:

class TreeMap<K, V>

- **K** – type of keys
- **V** – type of values

Constructors of TreeMap

TreeMap()

TreeMap(Comparator<? super K> comp)

TreeMap(Map<? extends K, ? extends V> m)

TreeMap(SortedMap<K, ? extends V> sm)

- **TreeMap()** – creates an empty tree map sorted by **natural order of keys**.
- **TreeMap(Comparator comp)** – creates an empty tree map sorted using the **specified comparator**.
- **TreeMap(Map m)** – creates a tree map initialized with entries from **map m**, sorted by natural key order.
- **TreeMap(SortedMap sm)** – creates a tree map with entries from **sorted map sm**, maintaining the same order.

Important Points

- TreeMap **extends AbstractMap** and **implements NavigableMap**.
- It **does not add new methods** beyond those defined by **NavigableMap and AbstractMap**.
- **Keys are automatically sorted in ascending order**.
- Sorting can be changed using a **Comparator** while creating the TreeMap.

LinkedHashMap Class

The **LinkedHashMap class** extends **HashMap**. It maintains a **linked list of entries in the order in which they were inserted**. This allows **insertion-order iteration** over the map. When iterating through a collection-view of a LinkedHashMap, the elements are returned **in the same order in which they were inserted**. It can also return elements **in the order in which they were last accessed**.

SBC Karkala

LinkedHashMap is a **generic class** and declared as:
class LinkedHashMap<K, V>

- **K** – type of keys
- **V** – type of values

Constructors of LinkedHashMap
LinkedHashMap()
LinkedHashMap(Map<? extends K, ? extends V> m)
LinkedHashMap(int capacity)
LinkedHashMap(int capacity, float fillRatio)
LinkedHashMap(int capacity, float fillRatio, boolean Order)

- **LinkedHashMap()** – creates a default LinkedHashMap.
- **LinkedHashMap(Map m)** – creates a map initialized with elements from m.
- **LinkedHashMap(int capacity)** – creates a map with specified capacity.
- **LinkedHashMap(int capacity, float fillRatio)** – creates a map with specified capacity and fill ratio.
- **LinkedHashMap(int capacity, float fillRatio, boolean Order)** – specifies whether elements are stored in **insertion order or access order**.

Default **capacity = 16** and **fill ratio = 0.75**.

Method in LinkedHashMap
protected boolean removeEldestEntry(Map.Entry<K, V> e)

- Called by **put()** and **putAll()**.
- The **oldest entry** is passed as e.
- By default, it returns **false** and does nothing.
- If overridden to return **true**, the **oldest entry is removed**.

IdentityHashMap Class
The **IdentityHashMap class** extends **AbstractMap** and implements the **Map interface**. It is similar to **HashMap**, but it uses **reference equality** when comparing elements.
It is declared as:
class IdentityHashMap<K, V>

- **K** – type of key
- **V** – type of value

The API documentation states that **IdentityHashMap is not for general use**.

EnumMap Class
The **EnumMap class** extends **AbstractMap** and implements **Map**. It is specifically used with **keys of enum type**.
It is declared as:
class EnumMap<K extends Enum<K>, V>
Here **K must extend Enum**, which ensures that keys are **enum types**.

SBC Karkala

Constructors of EnumMap

EnumMap(Class<K> kType)
EnumMap(Map<K, ? extends V> m)
EnumMap(EnumMap<K, ? extends V> em)

- **EnumMap(Class kType)** – creates an empty EnumMap of type kType.
- **EnumMap(Map m)** – creates an EnumMap containing the same entries as m.
- **EnumMap(EnumMap em)** – creates an EnumMap initialized with entries from em.

EnumMap **does not define any methods of its own**.

Comparators

Both **TreeSet** and **TreeMap** store elements in **sorted order**. The **Comparator** determines what the **sorted order** means.

By default, these classes use **natural ordering** (for example: A before B, 1 before 2). If a different ordering is required, a **Comparator** can be specified when creating the set or map.

Comparator is a **generic interface** declared as:

interface Comparator<T>

Here, **T** specifies the type of objects being compared.

Methods of Comparator

1. **compare()**

int compare(T obj1, T obj2)

- Compares two objects.
- Returns **0** if objects are equal.
- Returns a **positive value** if obj1 is greater than obj2.
- Returns a **negative value** if obj1 is less than obj2.
- It may throw **ClassCastException** if objects are incompatible.

By overriding this method, the **sorting order can be changed**, such as sorting in reverse order.

2. **equals()**

boolean equals(Object obj)

- Tests whether an object is equal to the invoking comparator.
- Returns **true** if both objects are Comparator objects with the same ordering.
- Otherwise returns **false**.
- Overriding this method is **usually unnecessary**.

Using a Comparator

A **Comparator** is used when we want to **define a custom sorting order** instead of the default natural order. It allows us to control **how elements are compared and stored in sorted collections and maps**.

SBC Karkala

Java Program using Custom Comparator for Reverse Order

```
import java.util.*;

// Reverse comparator for strings
class MyComp implements Comparator<String>
{
   public int compare(String a, String b)
   {
      return b.compareTo(a);   // reverse comparison
   }
}

class CompDemo
{
   public static void main(String args[])
   {
      TreeSet<String> ts = new TreeSet<String>(new MyComp());

      ts.add("C");
      ts.add("A");
      ts.add("B");
      ts.add("E");
      ts.add("F");
      ts.add("D");

      for(String element : ts)
         System.out.print(element + " ");
   }
}
```

Output

F E D C B A

Explanation

The **Comparator class overrides compare()** to reverse the normal comparison.
Because b.compareTo(a) is used instead of a.compareTo(b), the **TreeSet stores elements in reverse order**.

Arrays Class

The **Arrays class** provides various **utility methods for working with arrays**. These methods help bridge the gap between **collections and arrays**.

Important Methods of Arrays

1. asList()

static <T> List asList(T... array)

- Returns a **List backed by the specified array**.
- The **array and list refer to the same location**.

SBC Karkala

2. binarySearch()

static int binarySearch(int array[], int value)

- Searches a **value in a sorted array using binary search**.
- Returns the **index if found**, otherwise returns a **negative value**.

Overloaded forms/signatures of **binarySearch()**

static int binarySearch(int array[], int value)
static int binarySearch(byte array[], byte value)
static int binarySearch(long array[], long value)
static int binarySearch(Object array[], Object value)

---

3. copyOf()

static int[] copyOf(int source[], int len)

- Returns a **copy of an array**.
- If the new array is longer, extra elements are **filled with default values**.

---

4. copyOfRange()

static int[] copyOfRange(int source[], int start, int end)

- Returns a **copy of a specified range of elements** from an array.

---

5. equals()

static boolean equals(int array1[], int array2[])

- Returns **true if two arrays are equal**, otherwise **false**.

---

6. deepEquals()

static boolean deepEquals(Object[] a, Object[] b)

- Checks equality of **arrays including nested arrays**.

---

7. fill()

static void fill(int array[], int value)

- **Assigns a specified value to all elements** in the array.

---

8. sort()

static void sort(int array[])

- **Sorts array elements in ascending order**.

Difference between ArrayList and Arrays

ArrayList	**Arrays**
Size is **dynamic** (can grow or shrink).	Size is **fixed** once created.
Part of **Java Collections Framework**.	Part of **core Java language**.
Stores **objects only**.	Can store **primitive types and objects**.
Provides many **built-in methods** like add(), remove().	Limited operations available.

SBC Karkala

Difference between Vector and Arrays

Vector	Arrays
Dynamic size (can grow automatically).	**Fixed size** once created.
Part of **Java Collections Framework**.	Built-in **data structure** in Java.
Provides methods like **add(), remove(), size()**.	No built-in methods for insertion or deletion.
Stores **objects only**.	Stores **primitive types and objects**.

Program to Convert Array to Collection using asList()

```
import java.util.*;

class ArrayToCollection
{
   public static void main(String args[])
   {
      // Create an array
      String arr[] = {"Java", "Python", "C++", "C"};

      // Convert array to List using asList()
      List<String> list = Arrays.asList(arr);

      // Display the collection
      System.out.println("Elements in the List:");
      for(String s : list)
      {
         System.out.println(s);
      }
   }
}
```

Output:

```
Elements in the List:
Java
Python
C++
C
```

- An **array of strings** is created.
- Arrays.asList() converts the **array into a List (collection)**.
- The elements of the list are displayed using a **for-each loop**.

SBC Karkala